

Live Forex Volatility Tracker

A complete guide: the concepts, the decisions, and the code

AUTHOR	Vaibhav Yadav
REPOSITORY	github.com/VaibhavYDev/live-forex-volatility-tracker
SCOPE	Streaming FX volatility with regime detection — one upstream connection across N replicas, a Redis write-ahead log, $O(1)$ volatility mathematics, and a browser that absorbs 50 messages/second without putting React on the 60 Hz path
CODEBASE	~6,700 lines Python · ~7,000 lines TypeScript · 287 backend tests · 251 frontend tests
HOW TO READ	Part I needs no prior knowledge. Part II assumes Part I. Part III is a reference map.

Part I · The Concepts

1. The market, in the only detail that matters here
2. What "volatility" actually means
3. Computing variance without destroying it
4. $O(1)$ is the whole game
5. Five ways to measure the same thing
6. From a number to a decision
7. Hysteresis: why one threshold is never enough
8. Putting Part I together

Part II · The Decisions

9. The shape of the system
10. Exactly one writer
11. Redis as a write-ahead log
12. $O(1)$ as an architectural commitment
13. Backpressure without unbounded queues
14. Failure handling, and the bugs that taught it
15. Failure modes, stated in advance
16. The browser is a real-time system too
17. Testing: the part most projects skip
18. Honesty as a design constraint

Part III · The Code

19. Repository layout
20. A tick's journey, end to end
21. packages/core — the domain
22. apps/ingestor — the only writer
23. apps/api — fan-out
24. apps/web — the dashboard
25. Running it
26. Where each concept lives
27. Glossary
28. References
29. If you are asked one question

Part I · The Concepts

This part assumes nothing. If you already know what a pip is, skim §1 and start at §3 — that is where the first genuinely non-obvious idea appears.

1. The market, in the only detail that matters here

1.1 What is actually being traded

A foreign-exchange **pair** like `EURUSD` is a price for one currency expressed in another. `EURUSD = 1.0850` means one euro costs 1.0850 US dollars. The first currency is the **base**, the second the **quote**.

FX is the largest market in the world and it has no single exchange. There is no central order book, no opening bell, no authoritative last price. Banks and brokers quote prices to each other continuously, and what you receive from a data vendor is that vendor's view, assembled from whichever counterparties it can see.

Two consequences shape everything in this project:

There is no canonical price. Two vendors will disagree in the last decimal place, always. Any system that treats one feed as ground truth is making a claim it cannot support.

The market is open 24 hours, five days a week. It opens Sunday evening in Sydney and closes Friday evening in New York. That weekend gap is not an outage; it is the market being shut, and confusing the two is a defect we will return to several times.

1.2 Ticks, bid, ask, spread

A **tick** is one price update. It carries at minimum:

FIELD	MEANING
<code>bid</code>	the best price someone will <i>buy</i> from you at
<code>ask</code>	the best price someone will <i>sell</i> to you at
<code>mid</code>	<code>(bid + ask) / 2</code> — a synthetic reference price, nobody trades at it
<code>ts</code>	the timestamp the vendor stamped it with

The **spread** is `ask - bid`. It is the dealer's margin and the round-trip cost of entering and leaving a position. It widens when the market is uncertain, so it is itself a rough volatility signal.

A **pip** is the conventional smallest quoted increment. For most pairs that is 0.0001; for yen pairs it is 0.01, because a yen is worth roughly a hundredth of a dollar and quoting to four decimals would be absurd. This is why the codebase carries a `pip` per symbol rather than a global constant:

```

SYMBOL_DEFAULTS = {
    "EURUSD": _SymbolSpec(1.0850, 0.070, 0.0001, 0.6),
    "USDJPY": _SymbolSpec(147.20, 0.095, 0.01, 0.8),
    #           price   vol   pip   spread(pips)
}

```

and why the dashboard formats yen pairs to three decimals and everything else to five:

```
const dp = symbol.endsWith("JPY") ? 3 : 5;
```

Small detail, but a dashboard that prints `147.18300` for USDJPY announces immediately that nobody who trades FX has looked at it.

1.3 Ticks are not evenly spaced

This is the first thing that breaks naïve implementations. Ticks arrive when the market produces them: dozens per second during the London/New York overlap, one every few seconds at 3am Tokyo time. Any statistic computed "per tick" is therefore measuring a quantity whose sampling rate varies by an order of magnitude over a day.

The standard fix, and the one this project uses, is to **bucket** ticks into fixed time bars — here, one minute — and compute statistics per bar. A bar summarises all ticks that fell inside its minute:

FIELD	MEANING
<code>o</code>	open — first price in the bucket
<code>h</code>	high — maximum
<code>l</code>	low — minimum
<code>c</code>	close — last price in the bucket
<code>n</code>	tick count

This is the familiar **OHLC** candle. It is a lossy summary — you cannot reconstruct the path from it — but it makes every bar comparable to every other, which is the property the statistics need.

2. What "volatility" actually means

Volatility is the **standard deviation of returns**. Everything else is detail about which returns, over what window, and expressed in what units.

2.1 Why log returns

The naïve return is a percentage change:

$$r = (P_t - P_{t-1}) / P_{t-1}$$

The **log return** is:

$$r = \ln(P_t / P_{t-1})$$

Three reasons the log version is used throughout this codebase.

They add. The log return over two periods is the sum of the two log returns. Simple returns do not compose that way — a +10% followed by a −10% leaves you at 0.99, not 1.00. Additivity is what lets a one-hour variance be built from sixty one-minute pieces, which is the entire basis of the $O(1)$ design in §4.

They are symmetric. A move from 1.00 to 1.10 and back is +0.0953 then −0.0953. In simple returns it is +10% then −9.09%, and a naïve average of those two is not zero. Asymmetry in the units becomes bias in the statistic.

Prices cannot go negative, and logs respect that. A log return of any magnitude maps back to a positive price. This matters when generating synthetic data, and it is exactly the bug that mutation testing caught in this project's backfill generator — see Part II §14.2.

2.2 Realised vs implied

Realised (or historical) volatility is measured from prices that already happened. That is what this project computes.

Implied volatility is extracted from option prices — it is the market's forecast. Different thing entirely, mentioned here only so you can answer the question if it is asked.

2.3 Annualisation, and why an unlabelled σ is meaningless

A standard deviation computed on one-minute log returns is a *per-minute* number, and it will be tiny — around 0.0001. Nobody quotes volatility that way. The convention is to scale it to an annual figure.

Because variance is additive in time and independent across periods, standard deviation scales with the **square root** of time:

$$\sigma_{\text{annual}} = \sigma_{\text{bar}} \times \sqrt{(\text{bars per year})}$$

This project uses a 24-hour trading day and 252 trading days:

```
SECONDS_PER_TRADING_YEAR = 252 * 24 * 3600    # 21,772,800
# one-minute bars per year = 252 * 24 * 60      = 362,880
```

So a per-minute σ of 0.0001 becomes $0.0001 \times \sqrt{362880} \approx 0.060$, i.e. **6% annualised** — a plausible figure for EURUSD.

The choice of 252×24 is a *convention*, not a fact. Equity systems typically use 252×6.5 hours. If you compare a number from this system against one computed on a different basis, you are comparing nothing. That is why the dashboard footer says:

σ is labelled with its estimator, window and annualisation basis (252 trading days $\times$ 24h = 362,880 one-minute bars/year). An unlabelled σ is meaningless.

The `√dt` scaling is also the single most consequential line in the synthetic data generator. Getting it wrong — using `dt` instead of `√dt` — produces a number wrong by a factor of roughly 250, and it is the mutation the test suite was written to catch:

```
sigma = annual_vol * math.sqrt(seconds / SECONDS_PER_TRADING_YEAR)
```

3. Computing variance without destroying it

3.1 The formula you were taught, and why it fails

Every statistics course gives you the "computational" formula:

$$\text{Var}(x) = E[x^2] - (E[x])^2$$

It is algebraically correct and numerically catastrophic. It computes a small number as the difference of two large, nearly equal ones — the textbook setup for **catastrophic cancellation**, where the significant digits annihilate and what survives is rounding error.

A concrete demonstration. Take the values `1e9`, `1e9 + 1`, `1e9 + 2`. The true variance is exactly 1. In IEEE 754 double precision the two-pass formula can return 0, or a negative number, depending on the order of operations. Negative variance then produces `NaN` when you take the square root, and a `NaN` in an exponentially weighted recursion never leaves — it poisons every subsequent value forever, because there is no window for it to age out of.

3.2 Welford's algorithm

Welford (1962) computes the same quantity in one pass, updating the mean and the sum of squared deviations together:

```
def update(self, x: float) -> None:
    self.n += 1
    delta = x - self.mean
    self.mean += delta / self.n
    # NOTE: the second term uses the *updated* mean. Using the old mean here
    # is the classic off-by-one that silently biases the variance.
    self.m2 += delta * (x - self.mean)
```

Read the last line carefully, because the comment is doing real work. `delta` was computed against the **old** mean; `(x - self.mean)` uses the **new** one. The product of those two is what makes the recursion exact. Using the same mean twice is a bug that produces plausible, slightly wrong numbers — the worst kind.

Variance is then $m^2 / (n - 1)$. The $n - 1$ is **Bessel's correction**: dividing by n gives the variance of the sample you happen to have; dividing by $n - 1$ gives an unbiased estimate of the variance of the population it was drawn from. You have a sample.

3.3 Merging accumulators — why an hour is sixty minutes of arithmetic

Chan, Golub and LeVeque (1979) showed that two Welford accumulators can be combined without revisiting the underlying data:

```
def merge(self, other: Welford) -> Welford:
    """Chan et al.'s parallel combination.

    This is what lets a 1h window be the *sum* of sixty 1m accumulators rather
    than a re-scan of every tick. The  $\text{delta}^2 * n_a * n_b / n$  correction term
    is the whole trick: it accounts for the two sub-samples having different
    means.
    """
```

This is why `Bar` in this project carries `sum_ret` and `sum_ret_sq` alongside its OHLC. Those are the sufficient statistics. A one-hour bar is the *merge* of sixty one-minute bars — pure addition, no re-scan, no stored tick history.

That property is what makes the rollout job trivial and the memory bounded, and it is the reason the benchmark in Part II §12 shows a flat cost per tick after 900,000 ticks.

4. $O(1)$ is the whole game

4.1 The rolling window trap

The obvious way to compute "volatility over the last hour" is to keep the last hour of returns in a list, and recompute the standard deviation on every tick.

This works. It passes every test you are likely to write. It is also **$O(n)$ per tick**, and it degrades in a way that is nearly impossible to catch in development: at 25 ticks/second the window holds 90,000 entries after an hour, so every tick triggers a 90,000-element scan. On a laptop, for the ten minutes you run it, everything is fine.

The failure appears after hours of uptime, in production, gradually.

4.2 EWMA — exponentially weighted moving average

The alternative keeps no history at all:

```
self.variance = self.lam * self.variance + (1.0 - self.lam) * ret * ret
```

One multiply, one add, constant memory, constant time. λ (λ) is the decay factor: each new observation gets weight $(1 - \lambda)$ and everything before it is scaled by λ . With $\lambda = 0.97$, the most recent bar contributes 3%, the one before 2.9%, and so on — an exponentially decaying tail with no hard cutoff.

The **half-life** — how many bars until a weight halves — is the intuitive parameter, and the code converts between them:

```
def lambda_for_halflife(halflife_bars: float) -> float:
    """Decay factor whose weight halves after halflife_bars observations."""
    return math.exp(-math.log(2.0) / halflife_bars)
```

$\lambda = 0.97$ corresponds to a half-life of about 23 bars.

4.3 Two subtleties in that recursion

Both are in the code with comments explaining them, and both are the kind of detail an interviewer probes.

Seeding. The first observation cannot be blended into a variance of zero, or the reported σ ramps up from meaninglessly small over many bars:

```
if self.n == 0:
    # Seeding with r^2 rather than 0 avoids a long warm-up ramp where the
    # reported sigma is meaninglessly small.
    self.variance = ret * ret
```

NaN poisoning. A single non-finite return would corrupt the recursion permanently, because — unlike a rolling window — there is nothing for it to fall out of:

```
if not math.isfinite(ret):
    # A single NaN would permanently poison the recursion - there is no
    # window to age it out of. Drop it; the caller counts it.
    return self.variance
```

Dropping it and *counting* it is the honest choice: the data is not silently altered, and the counter is observable in metrics.

4.4 The drop-out artefact

There is a second, subtler reason to prefer EWMA here, and it belongs to the alerting logic rather than the arithmetic.

A simple moving average has a **drop-out artefact**. When a large observation falls off the back of the window, the average jumps discontinuously — not because anything happened in the market, but because of the shape of the window. If that average is the baseline an alert is measured against, the alert fires or clears for a reason that exists only inside your data structure.

An EWMA has no back edge, so it has no drop-out. Its weakness is the opposite — infinite memory, so a spike decays but never fully leaves — and that is handled separately by the baseline freeze described in §6.4.

5. Five ways to measure the same thing

The dashboard shows five volatility estimates side by side, over the same bars. They disagree, and the disagreement is the point: each uses a different subset of the information in a bar.

Recall that a bar gives you four prices — open, high, low, close. The naïve estimator uses one of them.

5.1 Close-to-close

```
rets = [ln(b.close / a.close) for a, b in pairwise(bars)]
var   = sum((r - mean)**2 for r in rets) / (n - 1)
```

This is the sample standard deviation of close-to-close log returns. It is the definition most people mean by "volatility", and it **throws away roughly 80% of the information in each bar** — the high and the low are ignored entirely.

Consider two bars that open and close at exactly the same price. One never moved. The other spiked 200 pips up, crashed 300 down, and recovered. Close-to-close scores them identically, at zero.

5.2 Parkinson (1980)

```
σ² = (1 / (4 ln 2)) · mean( ln(H/L)² )
```

Uses the bar's **range**. Under the assumption of driftless geometric Brownian motion, the expected squared log-range relates to the variance by that `4 ln 2` constant. Parkinson is roughly **4.7× more efficient** than close-to-close — it extracts the same precision from about a fifth as many bars.

Its weakness: it assumes zero drift, and it cannot see gaps between bars.

5.3 Garman–Klass (1980)

```
σ² = mean( 0.5·ln(H/L)² - (2 ln2 - 1)·ln(C/O)² )
```

Combines range with the open-to-close move. About **6.6× more efficient** than close-to-close, and the most efficient of the simple estimators — but it still assumes zero drift.

Note the implementation detail, which is a genuine numerical subtlety:

```
# The per-bar term can go negative on a bar with a tiny range and a large
# open-close move; clamping the *mean* rather than each term preserves the
# estimator's unbiasedness.
return math.sqrt(max(total / len(bars), 0.0))
```

Clamping each term at zero before averaging would bias the estimator upward. Clamping the mean does not.

5.4 Rogers–Satchell (1991)

$$\sigma^2 = \text{mean}(\ln(H/C) \cdot \ln(H/O) + \ln(L/C) \cdot \ln(L/O))$$

The important property: it is **drift-independent**. Parkinson and Garman–Klass both assume the price has no trend, and both overstate volatility when it does. Rogers–Satchell stays unbiased on a trending pair — which is precisely when somebody cares about the number.

5.5 Yang–Zhang (2000)

$$\sigma^2 = \sigma^2_{\text{overnight}} + k \cdot \sigma^2_{\text{open-close}} + (1 - k) \cdot \sigma^2_{\text{Rogers-Satchell}}$$

The only one of the five that accounts for the **gap between one bar's close and the next bar's open**. In FX that gap is the weekend, every single week. `k` is chosen to minimise the estimator's variance.

This is the honest default for FX, and the code says so:

```
"yang_zhang": "Gap-aware. FX gaps every Friday 21:00 UTC, so this is the honest default."
```

5.6 Why show all five

Because a single number invites false confidence. When Parkinson reads 10.9% and close-to-close reads 6.7% over identical bars, that spread is information: it says the intrabar range is large relative to the close-to-close move, which is what a choppy, mean-reverting session looks like.

Showing one estimator hides that. Showing five, over the same window, with each labelled, is a claim you can defend.

6. From a number to a decision

Measuring volatility is arithmetic. Deciding that volatility is *unusual* is where the engineering starts.

6.1 Volatility clustering

Mandelbrot (1963) observed what every trader knows: **large changes tend to be followed by large changes, and small by small**. Volatility arrives in runs. It is not independent across time.

This has two consequences for us. It means a fixed threshold ("alert above 15% annualised") is useless, because the level that is normal for EURUSD is not the level that is normal for AUDUSD, and neither is constant across a week. And it means the baseline you compare against must itself adapt — which creates the problem in §6.4.

6.2 The z-score

The standard way to say "unusual" is a **z-score**: how many standard deviations is the current reading from its own trailing mean?

$$z = (x - \mu_{\text{baseline}}) / \sigma_{\text{baseline}}$$

A z of 3 is conventionally "1 in 740" under a normal distribution. This project escalates at $z \geq 3$.

6.3 Log space, and the measurement that justifies it

Here is the most quantitatively interesting decision in the project, and the one a reviewer with a statistics background will check first.

Realised volatility is not normally distributed. It is approximately *lognormal* (Andersen, Bollerslev, Diebold & Labys, JASA 2001) — it has a hard floor at zero and a long right tail. A z-score computed in linear space assumes symmetry, so "3 sigma" is nowhere near the rare event it advertises.

The project measured this rather than asserting it. On 60,000 bars of pure lognormal noise containing **no regime change at all**:

STATISTIC	P(Z ≥ 3.0)	VS. GAUSSIAN 0.135%	FALSE ALERTS/PAIR/DAY
linear z	1.783%	13.2×	~26
log-space z	0.159%	1.2×	~2.3

Twenty-six false alerts per pair per day is not an alerting system; it is a generator of noise that trains its users to ignore it. One `math.log` reduces that to roughly two.

There is a free bonus. In log space the statistic becomes **scale-invariant**: a doubling of volatility is the same z whether the pair sits at 5% or 15% annualised. One threshold therefore works across an entire watchlist, instead of one tuned constant per symbol.

6.4 The circularity problem, and the freeze

Now the subtle failure. The baseline is an EWMA of recent volatility. During a genuine volatility event, the readings feeding that EWMA are the elevated ones. The baseline therefore *rises to meet the crisis*, the z-score falls back toward zero, and the alert clears — while the market is still in turmoil.

The system would announce "back to normal" in the middle of the event.

The fix is to **freeze the baseline** while the regime is stressed, so the comparison stays against the calm period that preceded it. But an unconditional freeze has its own failure: if the market has genuinely re-based to a permanently higher level, a frozen baseline keeps the alert on forever.

So the freeze is **bounded** — it thaws after a maximum duration, and when it does, the transition back to normal is labelled with a *different cause*:

```
CAUSE_LABEL = {
    "threshold": "z-score crossed",
    "baseline_thaw": "re-baselined – the level was accepted, the market did not calm",
    "observation_lost": "feed unobservable – not a claim about the market",
}
```

Those three phrases are not decoration. "Back to normal" means three genuinely different things, and **only one of them is a statement about the market**:

- `threshold` — volatility actually fell. A market claim.
- `baseline_thaw` — volatility did not fall; we accepted the new level as the new normal. An *administrative* claim.
- `observation_lost` — the feed went away. A claim about our own plumbing, and emphatically not about the market.

A dashboard that renders all three as "normal" is lying by omission in two cases out of three.

6.5 Intraday seasonality

One more refinement. FX volatility has a strong daily shape: the London/New York overlap is reliably more active than the Tokyo afternoon. A z-score that ignores this will fire every day at 13:00 UTC and stay silent every day at 03:00 — detecting the clock, not the market.

The project carries a seasonality profile (`fx_core/alerts/seasonality.py`) so the comparison is against *this time of day's* normal, not the day's average.

7. Hysteresis: why one threshold is never enough

7.1 The chattering problem

Suppose you escalate when $z \geq 3$ and clear when $z < 3$. Now consider a z-score hovering right at 3.0, which is exactly what happens at the edge of an event.

It crosses. Alert. Drops below. Clear. Crosses. Alert.

At one bar per minute this could fire dozens of times an hour. Over a watchlist it is hundreds. The system is technically correct at every single step and completely useless.

7.2 The Schmitt trigger

The fix is borrowed from electronics: use **two** thresholds, with a gap between them.

```
@dataclass
class TriggerConfig:
    enter_z: float = 3.0
    exit_z: float = 1.5 # MUST be < enter_z; that gap IS the hysteresis band
```

- Below 1.5 → normal
- Above 3.0 → stressed
- **Between 1.5 and 3.0 → whatever it already was**

That dead zone is the hysteresis band. A z-score wobbling around 3.0 crosses once, then has to fall all the way to 1.5 before it can clear. The chatter is gone, and the cost is a deliberate lag on the clear — which is the correct trade, because a late all-clear is far cheaper than a hundred false ones.

The invariant is enforced at construction, not documented and hoped for:

```
if self.exit_z >= self.enter_z:
    raise ValueError(
        f"exit_z ({self.exit_z}) must be below enter_z ({self.enter_z}); "
        ...
    )
```

7.3 Asymmetric confirmation

There is one more asymmetry worth understanding. Escalation requires several consecutive samples above `enter_z`; clearing requires only one below `exit_z`.

That is deliberate. A false escalation wakes somebody up. A false clear is recoverable — the next bar re-escalates. When the two error types cost different amounts, the confirmation requirements should differ too.

7.4 Why the band is drawn on the chart

The dashboard renders the band as a shaded region between the two levels, not as a single line at 3.0:

The dead zone IS the mechanism. A single line would show the escalation level and hide the reason an alert does not immediately un-fire.

Someone looking at the z-score pane and wondering why an alert is still active at $z = 2.2$ has the answer on screen: 2.2 is inside the band.

8. Putting Part I together

You now have every concept the system rests on. In order:

1. Ticks arrive irregularly, so they are bucketed into one-minute OHLC bars.
2. Returns are taken in log space, so they add and stay symmetric.
3. Variance is accumulated with Welford, so it never suffers catastrophic cancellation, and with additive statistics so wider windows are a merge rather than a re-scan.
4. The fast signal is an EWMA — $O(1)$ in time and memory, with no drop-out artefact.

5. Five estimators are reported side by side, because each discards different information and the disagreement is itself informative.
6. "Unusual" is a z-score against a trailing baseline, computed in **log space** because volatility is lognormal — a change measured at $13.2\times$ fewer false positives.
7. The baseline **freezes** during stress so it cannot rise to meet the crisis, and **thaws** on a bound so it cannot hold an alert open forever — with the two outcomes labelled differently because they mean different things.
8. Transitions use **two thresholds**, and the gap between them is what stops a hovering z-score from firing a hundred times an hour.

Part II is about everything that surrounds this: how to run it without losing data, without stampeding an upstream provider, and without freezing a browser.

Part II · The Decisions

Part I was about what the system computes. This part is about why it is built the way it is. Every section states the problem, the decision, and — most importantly — **the alternative that was rejected and why**. That last part is what separates an answer from a recitation.

9. The shape of the system

Five services, one job each.



SERVICE	RESPONSIBILITY	REPLICAS
ingestor	one upstream connection; bucket ticks; run the detector	2, one active
redis	write-ahead log, hot cache, pub/sub, leader lease	1
worker	drain the WAL into Postgres	1
api	REST + WebSocket fan-out	1..N
web	static SPA + reverse proxy	1..N

The load-bearing idea is that **Redis sits between every pair of services**. No service calls another directly. That is what makes each independently restartable, and it is why the failure-mode matrix in §15 has so many rows that end in "degrades" rather than "outage".

10. Exactly one writer

10.1 The problem

Run two ingestor replicas for availability and you now have two upstream connections, two sets of bars for the same minute, and two detectors emitting transitions for the same event. Duplicate alerts, doubled tick counts, corrupted statistics.

Run one replica and a crash means no data until somebody notices.

10.2 The decision: a lease, not consensus

Exactly one replica holds a **lease** in Redis. Only the holder connects upstream. The others sit in standby and take over when the lease expires.

Acquisition is a single atomic operation:

```
ok = await self._redis.set(self._key, self.instance_id, nx=True, px=self._ttl_ms)
```

`nx=True` means "only if the key does not exist" — that is the mutual exclusion. `px` sets a TTL, so a leader that dies without releasing the key loses it automatically after `ttl_ms` (10 s by default).

Renewal cannot be a plain `SET`, and this is the detail worth knowing:

```
# and we would then extend *someone else's* lease. Lua runs atomically.
```

Between "check the lease is still mine" and "extend it" there is a window in which the lease could expire and be taken by another replica. A `GET` followed by a `SET` would then extend a lease that now belongs to someone else. The renewal is therefore a **compare-and-set in Lua**, which Redis executes atomically.

There is a guard on the configuration too:

```
if renew_ms >= ttl_ms / 2:
    raise ValueError(
        f"renew_ms ({renew_ms}) must be well below ttl_ms/2 ({ttl_ms / 2}); ..."
    )
```

Renew at 5 s with a 10 s TTL and a single dropped packet costs you the lease. The constraint is enforced at startup rather than discovered at 3am.

10.3 What was rejected, and the honest caveat

Raft or ZooKeeper would give real consensus. Rejected: an enormous dependency for a problem that does not need it.

And here is the part to volunteer rather than wait to be asked. **This is a lease, not consensus, and it cannot guarantee single-writer under adversarial conditions.** Martin Kleppmann's critique of Redlock applies directly: a leader that is paused — GC, VM suspend, a laptop lid closing — can wake believing it still holds a lease that has since expired and been taken by another replica. For a moment there are two leaders, and no amount of Lua fixes that.

The code says so:

```
# can produce two leaders for a moment, and no amount of Lua fixes that - it is the
```

The reason it is acceptable here is that the consequence is bounded: a few duplicate ticks in a WAL that is idempotent on `(symbol, seq)` downstream.

Knowing the limits of your own mechanism is worth more in an interview than the mechanism.

11. Redis as a write-ahead log

11.1 The decision

Ticks are appended to a Redis **Stream** before anything else happens to them. The worker consumes that stream and writes to Postgres.

Why not have the ingestor write to Postgres directly? Because then a Postgres restart, a slow query, or a lock stalls ingestion — and the market does not wait. The WAL decouples the two: ingestion continues at full speed while Postgres is down, and the worker catches up afterwards.

11.2 Trimming by time, not by count

```
XADD stream:ticks MINID ~ <now - retention>
```

`MAXLEN 1000000` would be the obvious choice. It is worse, and the reasoning generalises:

*"Keep 15 minutes of replay buffer" is a requirement you can put in a README and reason about during an outage.
"Keep 1,000,000 entries" means something different at 20 ticks/sec than at 2,000.*

A count-based bound silently changes meaning with load. A time-based bound does not.

The `~` makes trimming approximate — Redis drops whole macro-nodes rather than walking entries — which is what keeps `XADD` $O(1)$ amortised.

11.3 Consumer groups and the crash case

The worker reads through a **consumer group**, so a message is delivered to one consumer and stays pending until acknowledged. A worker that crashes mid-batch leaves its messages pending rather than losing them, and `XAUTOCLAIM` lets a replacement take them over.

11.4 The eviction trap

Redis's `maxmemory-policy` is **server-wide**, not per-key. Set it to `allkeys-lru` and under memory pressure Redis will happily evict your write-ahead log and your leader lease.

The project's answer is an invariant, enforced in the config file and encoded in `keys.py`:

```
DURABLE_KEYS: Final = frozenset({STREAM_TICKS, STREAM_ALERTS, LEADER_LEASE})
```

with `volatile-lru` as the policy: **a key with a TTL is disposable, a key without one is durable**. Every cache key carries a TTL; the WAL and the lease do not. The policy then cannot touch what matters.

12. O(1) as an architectural commitment

Part I §4 covered the mathematics. This is the measurement.

PROPERTY	MEASURED	WHY IT MATTERS
Volatility hot path, first 100k ticks	801 ns/tick	
Volatility hot path, after 900k ticks	788 ns/tick	ratio 0.98× — flat, i.e. O(1)
Sustainable rate implied	~1.2 M ticks/s on one core	upstream delivers 10–50/s
Publish throughput	1,506 k/s	

The number that matters is not 801 ns. It is the **ratio**: 0.98×, after nine times as much data. An O(n) implementation climbs without bound and degrades only after hours of uptime — which is to say, it passes every test you write and fails in production.

The benchmark asserts the *ratio*, and it does so with headroom:

*Run-to-run variance on a shared 1-vCPU container is real: the O(1) ratio has been observed between **0.98×** and **1.16×** across runs. The assertion has headroom for that, because a benchmark you cannot reproduce is worse than no benchmark.*

Stating measurement error rather than quoting the best run is the difference between a benchmark and marketing.

13. Backpressure without unbounded queues

13.1 The problem

One browser on hotel wifi must not be able to exhaust the memory of a server shared with everyone else. The naïve `await websocket.send_json(...)` is worse still: it blocks the event loop on the slowest client in the fan-out, so one bad connection degrades every other.

13.2 The decision: conflation

Each client session holds a **dictionary keyed by routing key**, not a queue:

```

def offer(self, key: str, payload: Any) -> None:
    """Non-blocking publish. This is the hot path; it must never await."""
    if self.closed:
        return
    if key in self.pending:
        # The previous value for this key was never read. Dropping it is the
        # point: the client is about to receive something strictly newer.
        self.conflated += 1
        FRAMES_CONFLATED.inc()
    self.pending[key] = payload
    self.wakeup.set()

```

If a client has not read the last EURUSD price and a newer one arrives, the older one is **overwritten**. Memory is bounded by the number of subscribed symbols, not by the message rate.

The measurement: **200,000 updates collapse to 5 entries held**. A naïve queue holds 200,000.

Fan-out to 500 clients with 450 of them not reading: **p50 312 µs, p99 533 µs**. In the naïve design that figure is whatever the slowest client's network decides.

13.3 The exception that proves the rule

Conflation is only valid for **last-value-wins** data. A price is; an event is not.

ALERTS MUST NOT BE CONFLATED.

Conflation is safe for prices because they are last-value-wins: a superseded tick is a stale price nobody wants. A regime transition is a discrete EVENT - "we escalated at 14:03" is not made redundant by "we cleared at 14:09".

So alerts bypass the conflating path entirely and are keyed by `seq`, which also gives idempotency on redelivery. Knowing *which* data a technique applies to is the actual skill; applying conflation uniformly would silently drop alerts.

13.4 The slow-client policy

A client that conflates occasionally is fine — that is the mechanism working. A client whose queue is *continuously* non-empty is not reading at all:

```

"""True when this client has been behind for longer than the deadline.

A client that conflates occasionally is fine - that is the mechanism
working. A client whose queue is *continuously* non-empty is not reading
at all (laptop asleep, connection black-holed) and is holding memory for
nothing. Disconnect it with an honest close code and let it reconnect.
"""

```

Saturated for more than 10 seconds → close with code `4003`. An honest close is better than an indefinite hold, and the client's own backoff handles the rest.

14. Failure handling, and the bugs that taught it

This section is the most useful one for an interview, because every item is a real defect found in this codebase, with the fix and the lesson.

14.1 The reconnect ladder — a self-inflicted DDoS

The bug. The retry counter reset on *any* clean return from the consume loop. A provider that accepts a connection and immediately hangs up therefore produced: connect, disconnect, reset to base delay, connect... roughly **four reconnects per second, indefinitely, against someone else's infrastructure**. Worse, the circuit breaker recorded a *success* on connect, so it stayed closed and never tripped.

The fix. A session shorter than a threshold did not work, whatever the socket said:

```
# A session that ended in milliseconds did not work, whatever the
# socket reported. Resetting the ladder on ANY clean return meant
# a provider stuck in accept-then-hangup was retried at the base
# delay forever - roughly four times a second, against someone
# else's infrastructure, with the breaker held closed by a
# success recorded on connect.
healthy = (self._monotonic() - started) >= MIN_HEALTHY_SESSION_S
```

The lesson. "Connected" is not "working". Health has to be defined by something the failure mode cannot fake — here, elapsed time.

14.2 Full jitter

Backoff is `uniform(0, min(cap, base * 2^attempt))` — **full jitter**, not exponential backoff with a small random addition.

The reason: without jitter, N clients disconnected by the same event retry at the same instant, forever, in lockstep. The thundering herd reconverges after every attempt. Randomising the *entire* interval spreads them permanently.

14.3 The feed heartbeat — a "Stale" banner over live prices

The bug. Feed status carried one timestamp, `ts`, stamped when the state *changed*. The heartbeat loop wrote the cached payload back every 5 s — original timestamp included. Every consumer read `ts` as "when did we last hear from the feed".

On a feed that connected once and streamed for hours, the two diverge without limit. The visible result was a dashboard reading "Stale · last update 1336s ago" over prices updating on screen.

The invisible result was worse: `/readyz` applies the same 60 s threshold, so **every API replica declared itself unready one minute after a healthy start and stayed that way**. Behind a load balancer that is a total outage caused by nothing being wrong. CI passed only because it polls for at most 60 seconds and won the race.

The fix. Two timestamps, because there are two questions:

- `ts` — "when did the ingestor last confirm this?" (liveness; re-stamped every heartbeat)
- `since` — "when did the state last change?" (so the UI can say "degraded for 12 minutes")

The lesson. When one field answers two questions, it will eventually answer neither. And a heartbeat must be **published**, not only stored — the stored key serves clients that connect later; only pub/sub reaches the tab that is already open.

14.4 rAF is outside React's stack

The bug. A single tick listener that threw took down the entire feed for every symbol.

Why. The store flushes inside `requestAnimationFrame`. An error boundary catches exceptions thrown during React's render and commit phases. A rAF callback is neither — it is a fresh stack the browser invokes directly, so the exception escapes React entirely, kills the flush loop, and every chart stops.

The fix. Isolate each listener:

```
for (const fn of fns) {
  try { fn(q); }
  catch (err) {
    this.stats.painterErrors++;
    console.error("[MarketStore] tick listener threw", err);
  }
}
```

The lesson. Error boundaries do not catch everything. Anything outside React's call stack — `setTimeout`, `requestAnimationFrame`, event handlers, promise rejections — needs its own guard.

14.5 Compose: a fixed host port on a scaled service

The bug. The ingestor declared `ports: ["9100:9100"]` alongside `deploy.replicas: 2`. Two containers cannot bind one host port. The second died with "port is already allocated", Compose aborted the whole run, and `web` — last in the dependency graph — never started.

The user-visible symptom was **"localhost:5173 does not load"**, which points nowhere near the cause.

The fix. No host port; Prometheus discovers both replicas by DNS instead.

The lesson. `docker compose config` validates *schema*, and a fixed host port on a scaled service is schema-valid. The collision exists only at runtime, on the second container. So the check had to be written by hand — and it now lives in `tests/unit/test_compose.py`, because the compose file is part of the product.

14.6 Mixed content — the silent one

The bug. The frontend hardcoded its API endpoints:

```
const WS_URL = `ws://${window.location.hostname}:8000/ws/stream`;
```

Hardcoded port, hardcoded **insecure** scheme. On any HTTPS deployment the browser blocks `ws://` as mixed content. The page renders perfectly, the socket never opens, the banner sits on "Connecting", and the only evidence is a console warning nobody reads.

The fix. Derive scheme from the page and stay same-origin:

```
const WS_SCHEME: Record<string, string> = { "https:": "wss:", "http:": "ws:" };
export function wsUrl(loc: Origin = window.location): string {
  return `${WS_SCHEME[loc.protocol]} ?? "ws:"`${loc.host}/ws/stream`;
}
```

Both nginx (production) and Vite (development) already proxied `/api` and `/ws`. **The infrastructure was correct on both ends; only the client bypassed it.**

The lesson. Silent failures deserve the most test coverage, because nothing else will find them.

14.7 `index.html` and the immutable-cache trap

The bug. nginx marked `/assets/` `immutable` for a year — correct, the filenames are content-hashed. It said nothing about `index.html`, so browsers cached it heuristically.

Those two rules combine into a trap. A returning visitor holds a stale `index.html`, requests an asset hash that no longer exists after a redeploy, and because `immutable` means "do not revalidate", never asks again. **Blank page.** One 404 in the log, and "a hard refresh mysteriously fixes it" as the only clue.

The fix. `index.html` is the *index* — it names the hashed bundles — so it must never be cached:

```
location = /index.html {
  add_header Cache-Control "no-cache, must-revalidate";
}
```

The lesson. Two individually correct caching rules can compose into a broken one.

14.8 The stale-feed watchdog, and not becoming the thing you fixed

The bug. A machine sleeps overnight. The ingestor stops. The WebSocket stays `OPEN`, because nothing in that layer noticed. The page sits on "Stale · last update 66237s ago" indefinitely, with a connection it considers healthy.

Two sub-bugs, both instructive:

The age readout was computed **once per store update** — and a stopped feed produces no more updates. So the number froze at the moment it started mattering. It now ticks on a one-second interval.

And the first fix for the reconnect was itself wrong: it rebuilt the socket every 15 seconds, forever, against a dead feed. **That is §14.1 reintroduced in the browser.** The corrected version backs off exponentially (15 s → 5 min), ignores hidden tabs — browsers throttle background timers, so a waking laptop surfaces a huge apparent age at once — resets on recovery, and lets the manual Refresh button bypass the backoff entirely, because a person who clicked is asking *now*.

The lesson. The same failure mode recurs at every layer. Recognising the shape is worth more than memorising any one instance.

15. Failure modes, stated in advance

WHAT BREAKS	DETECTED BY	BEHAVIOUR	DATA LOSS	RECOVERY
Upstream provider drops	staleness watchdog	detector disarmed, banner degrades	none	jittered backoff
Provider returns 401	<code>ProviderAuthError</code>	exit 78 , no retry	none	operator fixes key
Leader ingestor dies	lease TTL expires	standby promotes, rehydrates detectors	≤ lease TTL	automatic
Redis restarts	connection error	ingestion pauses, API serves cache	WAL beyond retention	automatic
Postgres down	worker write fails	messages stay pending in the WAL	none	worker catches up
Browser cannot keep up	queue saturated > 10 s	conflate, then close <code>4003</code>	superseded prices only	client reconnects
Tab backgrounded	rAF stops firing	buffer conflates	none	on focus

Two of these deserve a note.

A 401 exits rather than retries. An infinite backoff loop on a credential failure looks healthy on a dashboard while ingesting nothing, and will get the API key banned. Exit code 78 is `EX_CONFIG` — "this is a configuration problem, not a crash".

A promoted standby rehydrates before its first tick. Otherwise it re-baselines on whatever the market is doing *right now*: promoted mid-crisis, it learns the crisis as normal, reports NORMAL for the rest of the event, and never emits the clear that the stored `stressed` row is waiting for. That was measured in `tests/chaos/test_regime_pipeline.py` — and the failure was *not* the duplicate escalation that had been assumed, which would have been noisier and far kinder.

16. The browser is a real-time system too

16.1 The budget

60 fps means **16.7 ms per frame** for everything: your JavaScript, style, layout, paint, composite. The store is one tenant of that budget.

SCENARIO	MEAN	P99	% OF ONE FRAME
50 ticks across 5 pairs, 1 subscriber each	2.0 μ s	3.5 μ s	0.012 %
500-tick burst conflated into a single frame	9.7–12.0 μ s	38.6 μ s	0.058–0.072 %
50 ticks with 100 subscribers	7.7 μ s	38.9 μ s	0.046 %

Ten times the messages costs roughly four times the work, not ten. **That sub-linearity is the property worth quoting;** the absolute figures are small enough to be uninteresting on their own.

16.2 Why not just use React state

Because React is excellent at structural change and the wrong tool for a pixel stream. Routing 50 messages/second through `useState` puts the reconciler on the 60 fps path for no benefit.

The split this project uses:

- **React owns** the container, the lifecycle, and anything structural.
- **The chart owns its own pixels** — `series.update()` writes straight to canvas.

16.3 `useSyncExternalStore` and the `getSnapshot` rule

Components subscribe per topic — `tick:EURUSD` , `vol:EURUSD` , `status` — so a volatility update re-renders the panel and nothing else.

The rule that makes it work, and the one that bites everyone:

`getSnapshot` must return a stored reference and must never allocate.

React calls `getSnapshot` during render and compares the result with `Object.is` . Return a fresh object or array each time and you get an infinite re-render loop. Every accessor therefore hands back a frozen, cached reference, and there are shared empty constants (`NO_ALERTS` , `NO_Z`) so the empty case does not allocate either.

This is not theoretical: the same rule caught a real defect in the demo build, where a helper mapped an array on every call. Called during render, it handed the chart a new `bars` reference every time, and the snapshot effect keyed on `[bars]` rebuilt the entire series continuously.

16.4 rAF idle scheduling

Updates are buffered and flushed once per animation frame. Fifty messages in a frame become one flush, and one paint. A backgrounded tab stops firing rAF entirely, so the buffer simply conflates until focus returns — for free.

17. Testing: the part most projects skip

17.1 Four tiers

TIER	COUNT	NEEDS SERVICES	PURPOSE
tests/unit	192	no	pure logic, maths, config invariants
tests/integration	76	Redis, Postgres	real round trips
tests/chaos	19	yes	kill things and assert recovery
tests/load	—	yes	throughput and the O(1) ratio

Plus **251 frontend tests** at 90.4% statement coverage. 287 backend tests in total.

17.2 The silent-skip problem

Integration tests skip when Redis is unavailable. Skipped tests are green. A CI run where the whole integration tier silently skipped is indistinguishable from one where it passed.

The fix is an environment flag that turns skips into failures:

```
FX_REQUIRE_SERVICES=1
```

CI sets it. Local runs do not. This caught a real outage mid-session when Redis died in the dev container.

17.3 Mutation testing, and tests that pass for the wrong reason

The technique used throughout: **deliberately break the code and confirm the test fails**. A test that passes against a broken implementation is not a test.

Applied to every load-bearing guarantee in this project — around twenty mutations. Several found tests that passed for the wrong reason. Three examples, because they generalise:

A positivity check could not tell GBM from an additive walk. The backfill generator uses geometric Brownian motion. The test asserted prices stay positive — which an *additive* walk also satisfies, because per-minute sigma is $\sim 1.2e-4$ and 500 additive steps from 1.085 never reach zero. The property that actually separates them is **scale invariance**: under GBM a ¥147 pair and a \$1.08 pair have the same *relative* volatility; additively, the yen pair's would be $\sim 135\times$ smaller. That is the test now.

"Range > 0" does not imply a wick. Removing the wick entirely leaves `high == max(open, close)`, which still satisfies `high > low` on any bar that moved. But such a candle renders flat-topped, and Parkinson — which reads the high-low range — systematically understates volatility on it. The test now asserts that >90% of bars have wicks on both sides.

A single marker cannot test sort order. A test asserting chart markers were in ascending time passed against a *removed* sort, because the fixture produced only one marker. It now places an alert inside a stretch that guarantees the boundary marker lands after it.

17.4 The compose file is part of the product

`tests/unit/test_compose.py` asserts things like "no scaled service claims a fixed host port", "every service restarts itself", "every service caps its logs", "certificates survive a restart". These are not code, and they broke the product anyway (§14.5). If the README's first instruction is `docker compose up -d`, that file is user-facing and deserves tests.

18. Honesty as a design constraint

A theme worth naming explicitly, because it recurs and it is unusual enough to be memorable.

Every σ carries its estimator, window and annualisation basis. An unlabelled σ is meaningless — 0.004 could be per-tick, per-minute or annualised.

Backfilled bars are labelled and rendered differently. A marker shows where synthesised history ends and observed data begins, and a text note states how much of the window is synthetic. Nothing downstream can mistake one for the other.

"Market closed" is styled neutrally, not as an error. Silence at 3am on a Sunday is the FX market being shut, not a fault, and conflating the two trains people to ignore the banner.

Warming-up readings are greyed out. Serving a σ computed from three ticks as though it were a measurement is how dashboards lie.

Three different "back to normal" causes are labelled separately, because only one of them is a claim about the market (§6.4).

The benchmarks list what has *not* been measured. End-to-end p99 from provider timestamp to browser paint, multi-hour soak behaviour, the Postgres write ceiling. Publishing the headline number before measuring it properly would be dishonest, and listing the gaps is part of the benchmark.

The unifying principle: **a system that looks authoritative while being wrong is worse than one that shows nothing.** Most of the defects in §14 were failures of exactly that kind — the software kept rendering confidently while the thing behind it had stopped.

Part III · The Code

This part is a map. Read it once now, and again in six months when you have forgotten where everything lives.

19. Repository layout

```
live-forex-volatility-tracker/
├─ packages/
│  ├─ core/fx_core/           pure domain logic – no I/O, no Redis, no network
│  └─ platform/fx_platform/   cross-cutting: logging, metrics, Redis client
├─ apps/
│  ├─ ingestor/fx_ingestor/   upstream connection, bucketing, detection
│  ├─ worker/fx_worker/       WAL → TimescaleDB
│  ├─ api/fx_api/             REST + WebSocket fan-out
│  └─ web/                    React dashboard
├─ infra/
│  ├─ docker/                 Dockerfiles, nginx config
│  ├─ redis/redis.conf        eviction policy, and the reasoning
│  ├─ migrations/             schema, hypertables
│  ├─ caddy/Caddyfile         TLS for the hosted deployment
│  └─ grafana/                 dashboards, Prometheus scrape config
├─ tests/                     unit · integration · chaos · load
├─ scripts/build_demo_dataset.py
├─ docs/                      the published static build + runbooks
└─ docker-compose.yml         local stack (5 services)
    docker-compose.prod.yml    public profile (no database)
```

Roughly **6,700 lines of Python** and **7,000 of TypeScript**.

The most important structural rule: **packages/core imports nothing from apps/**. It has no Redis client, no HTTP, no async. That is what makes the mathematics unit-testable without services, and it is why **tests/unit** — 192 tests — needs no infrastructure at all.

20. A tick's journey, end to end

Follow one price update from the vendor to a pixel.

Step 1 — the provider yields it

apps/ingestor/fx_ingestor/providers/ holds one class per source behind a common interface: **replay** (deterministic synthetic, the default), **tiingo** (real).

`replay` exists so the project runs with no signup and CI can assert numerical results — a fixed seed makes the tick sequence reproducible.

Step 2 — the pipeline handles it

`fx_ingestor/pipeline.py`. Everything for one tick happens in **one Redis round trip**:

```
XADD    stream:ticks MINID ~ <now - retention>    durable WAL → Postgres
HSET    q:{SYMBOL}                                snapshot cache (TTL)
HSET    bar:{SYMBOL}:{bucket}                    in-progress bar (TTL)
PUBLISH ch:tick:{SYMBOL}                        fan-out to browsers
```

Done as four sequential `await`s that is four network round trips — roughly 8 ms of added latency. Pipelined into one `MULTI / EXEC` it is one, about 2 ms. At 25 ticks/sec the difference is invisible; at 2,500 it is the whole system.

Step 3 — a bar seals

When the clock crosses a minute boundary the accumulator closes and more writes join the same transaction:

```
ZADD    hist:{SYMBOL}:1m        recent bars for cold start (TTL)
ZADD    zhist:{SYMBOL}          z-score series for the pane (TTL)
HSET    vol:{SYMBOL}:1h         volatility snapshot (TTL)
PUBLISH ch:vol:{SYMBOL}         volatility fan-out
SET     regime:{SYMBOL}         detector state (DURABLE – no TTL)
```

Step 4 — the detector decides

The sealed bar's σ goes to `RegimeDetector.update()`. It computes a log-space z-score against the frozen-or-live baseline, passes it through the Schmitt trigger, and returns a `RegimeTransition` or `None`.

On the rare bars where the regime changes:

```
XADD    stream:alerts            durable WAL → Postgres
PUBLISH ch:alerts:{SYMBOL}      instant push to the browser
```

Both, in the same `MULTI`. Neither alone is sufficient — pub/sub is fire-and-forget and loses the alert if nobody is listening at that instant; the stream never loses it but a browser reading a consumer group per client would be absurd. So each does what it is good at, atomically: if the transition is recorded it is also broadcast, and vice versa.

Step 5 — the API fans it out

`fx_api/ws/manager.py` subscribes to the Redis channels once — not once per client — and dispatches into each session's conflating buffer (§13).

Feed status is special: it goes to **every** client regardless of subscription, because a user watching one pair still needs to know the whole feed is degraded.

Step 6 — the browser absorbs it

`MarketStore` buffers into a `Map` and flushes once per animation frame. Components subscribed to `tick:EURUSD` re-render; nothing else does. The chart's tick callback writes straight to canvas.

21. `packages/core` — the domain

The part with no dependencies and all the mathematics.

MODULE	CONTAINS
<code>models.py</code>	<code>Tick</code> , <code>Bar</code> , <code>VolSnapshot</code> , <code>BarSource</code> , <code>Estimator</code>
<code>keys.py</code>	every Redis key name and TTL, in one place
<code>volatility/welford.py</code>	streaming mean/variance, and <code>merge()</code>
<code>volatility/ewma.py</code>	<code>EwmaVariance</code> , <code>lambda_for_halflife</code>
<code>volatility/buckets.py</code>	<code>BucketAccumulator</code> , <code>RollingWindow</code> , <code>floor_to_bucket</code>
<code>volatility/range_estimators.py</code>	the five estimators
<code>alerts/baseline.py</code>	log-space EWMA baseline, freeze/thaw
<code>alerts/hysteresis.py</code>	<code>TriggerConfig</code> , the Schmitt trigger
<code>alerts/detector.py</code>	<code>RegimeDetector</code> — orchestrates the above
<code>alerts/seasonality.py</code>	intraday volatility profile
<code>calendar/fx_sessions.py</code>	market open/closed, session overlaps
<code>intervals.py</code>	timeframe roll-up (<code>fold</code> , <code>bucket_start</code>)
<code>backfill.py</code>	deterministic synthetic history

21.1 `models.Bar` — note the extra fields

```
class Bar:
    symbol: str
    bucket: datetime
    open: float
    high: float
    low: float
    close: float
    tick_count: int
    sum_ret: float          # ← sufficient statistics
    sum_ret_sq: float       # ←
    source: BarSource = BarSource.STREAM
```

`sum_ret` and `sum_ret_sq` are why a one-hour bar is `merge()` of sixty one-minute bars rather than a re-scan (Part I §3.3). `merge()` also propagates `BACKFILL` provenance: if either input was backfilled, so is the result.

21.2 `keys.py` — one file, no stray key names

Every Redis key in the system is constructed here.

```
STREAM_TICKS      = "stream:ticks"
STREAM_ALERTS     = "stream:alerts"
LEADER_LEASE      = "lock:ingestor:leader"
DURABLE_KEYS      = frozenset({STREAM_TICKS, STREAM_ALERTS, LEADER_LEASE})

TTL_QUOTE_S       = 300           # 5 min
TTL_BAR_S         = 3_600         # 1 h
TTL_VOL_S         = 300
TTL_HISTORY_S     = 86_400        # 24 h
TTL_ZHIST_S       = 86_400
TTL_STATUS_S      = 30
ZHIST_MAXLEN      = 1_440        # 24h of one-minute bars
```

The TTLs are not arbitrary — they encode the durability invariant from §11.4. `DURABLE_KEYS` has no TTL and therefore cannot be evicted under `volatile-lru`.

21.3 `intervals.py` — the timeframe roll-up

Two base series, because one cannot serve both ends:

```
1m  5m  15m 30m 1h  4h  ← folded from the stored 1-minute series
1d  1w  1M                ← folded from the stored daily series
```

A 200-candle weekly chart needs four years of history. At one-minute resolution that is 2.1 million bars — absurd to store, and unrenderable at 0.0004 px per candle. Real vendors split intraday and long-range for exactly this reason.

Every fixed interval divides 86,400 evenly, so buckets align to midnight UTC and never drift. Weeks and months cannot work that way — a month is not a number of seconds — so those bucket by calendar date:

```
if interval == "1w":
    monday = (at - timedelta(days=at.weekday())).replace(hour=0, ...)
```

And the roll-up rule that a wrong implementation gets subtly wrong:

```
cur["h"] = max(cur["h"], bar["h"])
cur["l"] = min(cur["l"], bar["l"])
cur["c"] = bar["c"]          # last close wins
# ...open comes from the FIRST bar in the bucket
```

OHLC does not average. Open is the first open, close is the last close, high and low are extrema. Getting open/close backwards produces a chart that looks plausible and is wrong on every candle.

22. `apps/ingestor` — the only writer

FILE	ROLE
<code>main.py</code>	lifecycle: lease → connect → stream → back off → retry
<code>leader.py</code>	<code>LeaderLease</code> — <code>SET NX PX</code> plus Lua CAS renewal
<code>pipeline.py</code>	tick → bar → Redis, in one round trip
<code>supervisor.py</code>	<code>BackoffPolicy</code> , <code>CircuitBreaker</code> , <code>StalenessWatchdog</code>
<code>providers/</code>	<code>replay</code> , <code>tiingo</code> , and the common base
<code>config.py</code>	pydantic settings, every knob with a default

`main.py` reads top to bottom as the lifecycle it implements:



Three details worth knowing.

The staleness watchdog disarms the detector before tearing the socket down. A frozen price reads as exactly zero volatility, so a detector still running during a dead feed reliably reports "volatility collapsed" — the most embarrassing false positive available to a market dashboard.

Rehydration happens before the first tick, not after (§15).

The heartbeat loop both re-stamps and publishes (§14.3).

23. `apps/api` — fan-out

FILE	ROLE
<code>main.py</code>	FastAPI app, CORS, metrics endpoint
<code>ws/manager.py</code>	one Redis subscription, N client sessions
<code>ws/conflator.py</code>	<code>ClientSession</code> — the conflating buffer
<code>ws/protocol.py</code>	the frame schema
<code>ws/tickets.py</code>	short-lived WebSocket auth tickets
<code>routers/market.py</code>	<code>/api/bars</code> , <code>/api/volatility</code> , <code>/compare</code>
<code>routers/health.py</code>	<code>/healthz</code> (alive), <code>/readyz</code> (ready)
<code>ratelimit.py</code>	Redis fixed-window limiter

The API opens no database connection. Every figure the dashboard renders is served from Redis. Postgres is the durability path — the worker drains the WAL into hypertables — not the dashboard path. That is what allows the production compose profile to omit the database entirely, and there is a test asserting the API imports no driver so the property cannot rot:

```
drivers = ("asyncpg", "psycpg", "sqlalchemy", "databases")
```

`/healthz` answers "is the process alive". `/readyz` answers "should traffic be routed here" — it checks Redis and the feed heartbeat age. Conflating the two is how a deploy takes down a service that was merely warming up.

24. apps/web — the dashboard

```
src/
├─ App.tsx                layout, tabs, landmarks, skip link
├─ components/
│   ├─ charts/PriceChart.tsx  candles, markers, regime shading
│   ├─ charts/ZScorePane.tsx  canvas z-line + hysteresis band
│   ├─ StatusBanner.tsx       feed health, refresh control
│   ├─ TimeframeBar.tsx       1m ... 1M selector
│   ├─ VolatilityPanel.tsx     the five estimators
│   ├─ RegimeTimeline.tsx     calm/stressed strip
│   ├─ ToastStack.tsx         transition notifications
│   └─ ErrorBoundary.tsx      per-region isolation
├─ lib/
│   ├─ stream/store.ts        MarketStore – the whole state layer
│   ├─ stream/client.ts       WebSocket with jittered backoff
│   ├─ stream/hooks.ts        useSyncExternalStore bindings
│   ├─ endpoints.ts           same-origin URL derivation
│   ├─ useChartBars.ts        per-timeframe data source
│   ├─ regime.ts              spans, stressed ranges, durations
│   ├─ ally.ts                live summaries, reduced motion
│   └─ theme.ts               palette, dark/light
└─ demo/                     serverless build: baked dataset + provider
```

24.1 MarketStore

The core of the frontend. Three responsibilities:

1. **Buffer** incoming ticks in a `Map` keyed by symbol.
2. **Flush** once per animation frame.
3. **Notify** only the topics that changed.

The `getSnapshot` rule from §16.3 governs every accessor. Listener isolation (§14.4) governs the flush.

24.2 Accessibility, concretely

Not an afterthought, and worth being able to point at:

- **WCAG 1.4.1** — regime is never colour alone. The badge has a shape, markers have distinct arrow directions, the selected timeframe carries weight and an underline as well as colour.
- **Canvas charts have visually-hidden live summaries.** A canvas is opaque to a screen reader, so each chart maintains a throttled text description — *"EURUSD 1.08491, up 0.12 percent across the window, regime normal."*
- **Roving tabindex** on both the symbol tabs and the nine timeframe buttons: one tab stop, arrow keys within.
- **Timeframe buttons carry spelled-out labels** — `1m` and `1M` are indistinguishable to a screen reader otherwise.
- **prefers-reduced-motion** disables transitions.

- **Toasts** use `role="alert"` for escalations and `role="status"` for clears, and escalations never auto-dismiss.

24.3 The demo build

`VITE_DEMO=1` swaps the WebSocket source for a baked dataset and changes nothing else.
`scripts/build_demo_dataset.py` runs the **real** `fx_core` engine — Welford, the estimators, `RegimeDetector`, `fold` — over a deterministic session and writes what it produced.

The alternative was porting the numerical core to TypeScript so the browser could compute it live. That would put a second implementation of the statistics in the repository, and the one on the public URL would be the one nobody tests.

The dataset is byte-deterministic, so rebuilding it in CI doubles as a check that the engine still produces the same numbers.

25. Running it

25.1 Locally

```
cp .env.example .env
docker compose up -d
```

Then <http://localhost:5173>. No API key: the default provider is `replay`.

Nothing is bind-mounted. Every service runs from a built image, so `up -d` on an existing image restarts the *old* binary without saying so. After editing source or pulling:

```
docker compose up -d --build
```

For iterating on the frontend, skip Docker entirely — `cd apps/web && npm run dev` hot-reloads on save.

25.2 The two deployment profiles

	DOCKER-COMPOSE.YML	DOCKER-COMPOSE.PROD.YML
Services	5 (+ Timescale, worker)	4, no database
Ports published	5173, 8000	80, 443 only
TLS	none	Caddy, automatic Let's Encrypt
Ingestor replicas	2	2

The production file is **standalone, not an overlay**. Compose merges `depends_on` key-by-key, so an overlay cannot *remove* the base file's dependency on Timescale — it would inherit a health gate on a service that profile deliberately does not run.

25.3 The published static build

`docs/` holds a pre-built demo bundle served by GitHub Pages from the branch. No server, no containers, no workflow. It is the only configuration that is up regardless of whether anything of yours is running.

26. Where each concept lives

A cross-reference back to Part I, for when you need to point at code.

CONCEPT	FILE
Log returns	<code>volatility/range_estimators.py::_ln</code>
Welford	<code>volatility/welford.py</code>
Accumulator merge	<code>volatility/welford.py::merge</code>
EWMA	<code>volatility/ewma.py</code>
Half-life $\leftrightarrow \lambda$	<code>volatility/ewma.py::lambda_for_halflife</code>
Five estimators	<code>volatility/range_estimators.py</code>
Annualisation	<code>backfill.py::SECONDS_PER_TRADING_YEAR</code>
Log-space z-score	<code>alerts/baseline.py</code>
Baseline freeze / thaw	<code>alerts/baseline.py::freeze</code>
Schmitt trigger	<code>alerts/hysteresis.py</code>
Intraday seasonality	<code>alerts/seasonality.py</code>
Timeframe roll-up	<code>intervals.py::fold</code>
Leader lease	<code>ingestor/leader.py</code>
Reconnect ladder	<code>ingestor/main.py::_run_as_leader</code>
Feed heartbeat	<code>ingestor/main.py::_heartbeat_loop</code>
Conflation	<code>api/ws/conflator.py::ClientSession.offer</code>
rAF flush + listener isolation	<code>web/src/lib/stream/store.ts</code>
Same-origin endpoints	<code>web/src/lib/endpoints.ts</code>
Stale-feed watchdog	<code>web/src/hooks/useForexStream.ts</code>

27. Glossary

Annualisation — scaling a per-period σ to a yearly figure by $\sqrt{(\text{periods}/\text{year})}$.

Backpressure — what a system does when it produces faster than a consumer reads. Here: conflation, then disconnection.

Bar / candle — OHLC summary of all ticks in a time bucket.

Conflation — replacing an unread update with a newer one. Valid only for last-value-wins data.

Catastrophic cancellation — loss of significant digits when subtracting two nearly equal floating-point numbers.

Consumer group — Redis Streams mechanism giving at-least-once delivery with per-consumer pending lists.

EWMA — exponentially weighted moving average; $O(1)$ memory, no drop-out.

Hysteresis — two thresholds with a dead zone between them, to stop chatter.

Idempotency — an operation safe to apply twice. Here: `UNIQUE (symbol, seq)`.

Jitter — randomising retry delays so clients do not reconverge.

Lease — a lock with a TTL. Not consensus.

Log return — $\ln(P_t / P_{t-1})$. Additive and symmetric.

Pip — smallest conventional price increment. 0.0001, or 0.01 for yen pairs.

Realised volatility — σ measured from past prices.

Regime — a state label: `normal` or `stressed`.

Schmitt trigger — the two-threshold comparator that gives hysteresis.

Spread — `ask - bid`.

Tick — one price update.

WAL — write-ahead log. Durable append before processing.

Welford's algorithm — numerically stable one-pass variance.

z-score — $(x - \mu) / \sigma$. Here computed in log space.

28. References

- Andersen, Bollerslev, Diebold & Labys (2001). *The distribution of realized exchange rate volatility*. JASA. — volatility is lognormal.
- Chan, Golub & LeVeque (1979). *Updating formulae and a pairwise algorithm for computing sample variances*. — the merge.
- Garman & Klass (1980). *On the estimation of security price volatilities from historical data*.
- Kleppmann (2016). *How to do distributed locking*. — why a lease is not consensus.
- Mandelbrot (1963). *The variation of certain speculative prices*. — clustering.
- Parkinson (1980). *The extreme value method for estimating the variance of the rate of return*.

- Rogers & Satchell (1991). *Estimating variance from high, low and closing prices*.
 - Welford (1962). *Note on a method for calculating corrected sums of squares and products*.
 - Yang & Zhang (2000). *Drift-independent volatility estimation based on high, low, open, and close prices*.
-

29. If you are asked one question

"What was the hardest bug?"

The feed heartbeat (§14.3). One field answered two different questions — "when did this state change" and "when did we last hear from the feed" — and on a healthy feed the two answers diverge without bound.

Visibly, it printed **"Stale · last update 1336s ago"** over prices updating on screen. Invisibly, `/readyz` used the same field, so every API replica declared itself unready one minute after a perfectly healthy start and stayed that way — behind a load balancer, a total outage caused by nothing being wrong. CI passed only because it polls for at most sixty seconds and won the race.

The fix was two fields with two names. The lesson is that when one value answers two questions, it will eventually answer neither — and that the loudest symptom was not the dangerous one.